Advanced database management systems
Topic 3: Data quality and master data management

Types of data at a company:

- Transactional (in OLTP databases)
- Hierarchical (taxonomies, data warehouses)
- Semi-structured (XML, json)
- Unstructured (email, pdf, blogs)
- Master data
- Metadata (data on data)

The requirement for a high Data quality (DQ) is stricter than that for data integrity (e.g. when we want to correct mistyped values).

Dimensions of data quality:

- Hard dimensions
    - Completeness—do we have all the data?
    - Accuracy—are the values accurate?
    - Consistency—do data contradict among different systems?
- Soft dimensions: as perceived by the users, e.g. trust.

If both soft and hard dimensions are evaluated low, a Master Data Management (MDM) solution should be applied. MDM requires a central data storage and *Data Governance*: the process of implementing data quality, and *Data Stewards*: people responsible for the quality of particular kinds of data, e.g. a steward may be responsible for customer data. **MDM technologies are not detailed in this course**.

As for Data quality (DQ), it can be improved in a planned or in a reactive way, the latter being applicable for a small company. The success of a DQ solution can be assessed by tracking e.g. how the number of erroneous/unknown records in a table decreases over time.

In the example below, we analyze the columns of tables for completeness, consistency, functional dependency etc.:

```
use AdventureworksDW2019[1]

--check if there is a functional dependency between two columns
select SalesReasonKey, SalesReasonName, SalesReasonReasonType from DimSalesReason

--the data suggests that the SalesReasonReasonType column depends on the
SalesReasonName column i.e. we can always tell the former from the latter
--check if the data supports this assumption

select SalesReasonReasonType, count(*) from DimSalesReason group by
SalesReasonReasonType  --3 groups with 4,5,1 members
select SalesReasonName, count(*) from DimSalesReason group by SalesReasonName  --each
group has 1 member -> candidate key
--conclusion: the SalesReasonName is likely a subcategory of the SalesReasonReasonType
```

[1] You can download this database from https://github.com/Microsoft/sql-server-samples/releases/download/adventureworks/AdventureWorksDW2019.bak

```
--checking functional dependency
--how to check if CountryRegionCode is functionally dependent on StateProvinceCode
use AdventureworksDW2016CTP3
select * from DimGeography
select StateProvinceCode, count(*) from DimGeography group by StateProvinceCode --71
select StateProvinceCode, count(*) from DimGeography group by
StateProvinceCode,CountryRegionCode  --71
--71=71 so a StateProvinceCode will always have the same CountryRegionCode
--it is highly likely that we have a functional dependency between the two fields
select color, count(*) from dimproduct group by color --10
select ProductLine, count(*) from dimproduct group by productline --5
select ProductLine, count(*) from dimproduct group by productline, color --27 > 5, 27
> 10
--there is no functional dependency between the two fields
```

A de facto functional dependency may be due to an unmodelled, but existing business rule of the business domain. Such a hidden relation may violate the 3NF structure via a transitive dependency within a table and it can lead to **redundancy and inconsistency**. Therefore it is the interest of the company to detect such relations by data analysis.

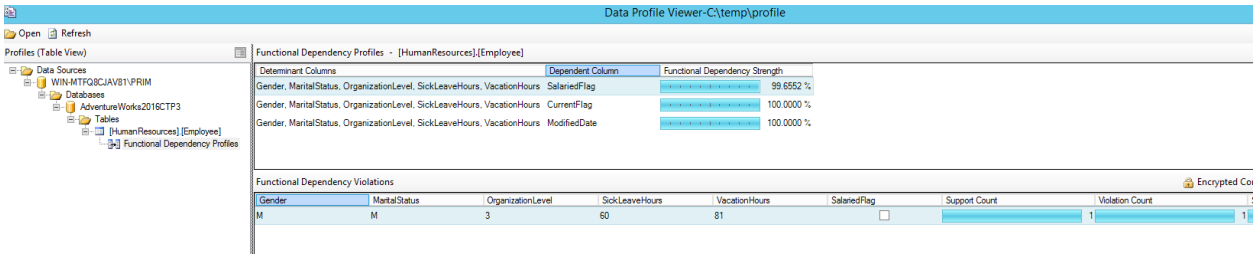
EXAMPLE: our employees have a job position code (like researcher, manager, technical or secretary) and an education level code (like elementary school, high school, Phd). There may exist a policy within the company that allows only a single education level for a certain job position, thus establishing *a de facto* functional dependency between the two fields. The education may depend on the job position.

Data profiling

Data Profiling or exploration means to automatically find statistics and data quality related features of a column:

- Identify functional dependencies, candidate keys and potential foreign keys (see above)
- Calculate column value distributions and other statistics
- Measure null ratio and length distribution for string types
- Derive *de facto* column patterns as regular expressions from the values that can be used as *domain rules* (see later)

In MS SQL Server, data profiling can be accomplished as an Integration Services Data Profiling Task that saves the results in an XML file (not detailed in this text)[2].



SQL Server Data Quality Services

The DQS Service is responsible for cleaning, profiling, and matching reference data. The DQS Server includes the DQS engine and the DQS databases:

[2] https://docs.microsoft.com/en-us/sql/integration-services/control-flow/data-profiling-task-and-viewer

- DQS_MAIN: stored procedures that implement the DQS engine, plus Knowledge Bases, e.g. US – Last Name knowledge base, reference data: e.g. cities in Hungary. Roles in the DQS_MAIN database: dqs_admininstrator, dqs_kb_editor, dqs_kb_operator (can be assigned to different users)
- DQS_PROJECTS: data needed for cleaning and matching projects
- DQS_STAGING_DATA: temporary storage for data to be cleaned, results of cleaning

Tasks to be performed by a Data Quality client: manage knowledge bases, execute cleaning projects, administer DQS. Client implementations:

- An SQL Server Integration Services 'DQS Cleansing transformation' node can perform cleaning in the data flow of an SSIS package
- SQL Server 2017 Data Quality Client (DQC), a desktop app

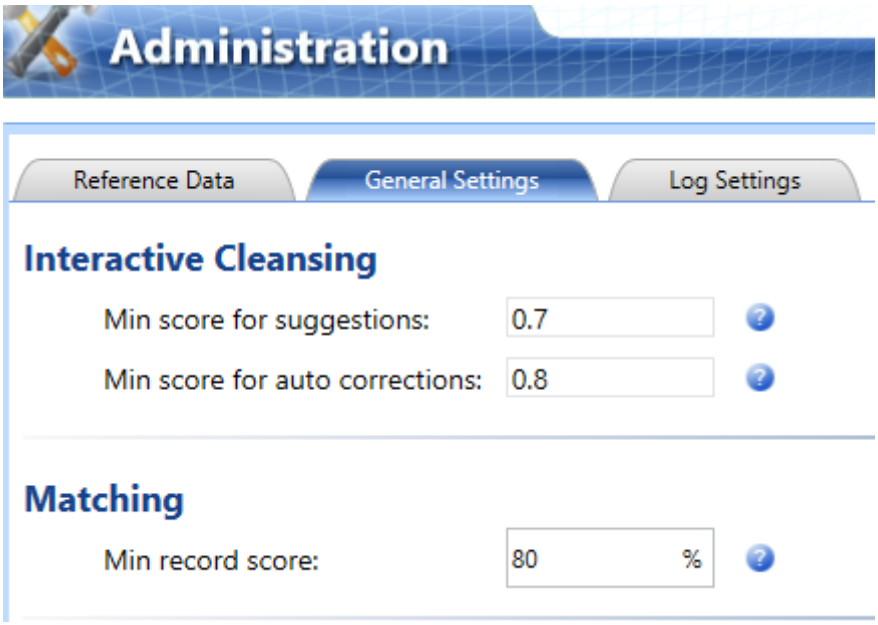
Data cleansing projects

There are two kinds of DQS projects:

- **Basic data cleansing**: the goal is to correct individual field values. We are going to demonstrate this type of project in this course.
- **Identity mapping and de-duplicating** according to matching rules: the goal is to identify and remove possibly duplicated records i.e. entities. The matching rule is defined at the KB level using common distance measures of data science. We are not going to demonstrate this type of project in this course notes.

Both types are based on a knowledge base (KB) already installed in the DQS_MAIN database, and use these parameters:

- Cleansing: the Min Score for Suggestions parameter sets the minimal similarity for generating a suggestion. It must be *less* than the Min Score for Auto Correction parameter.
- Matching (de-duplication): threshold value for the matching policy. It is a measure of record similarity.



A DQS Knowledge Base (KB) may contain several **domains** i.e. reference value sets used for cleansing. A domain is defined by the following components:

- **Name** and **data type** e.g. string
- Settings for normalization like converting to uppercase or removing dummy characters, formatting and spell checking
- Reference data, which may be stored in the DQS_MAIN database or alternatively, may be provided by an external service
    - The reference values, called **domain values**. A domain value is a tuple that contains a **leading value** and a list of its synonyms
- **Domain rules**, similar to CHECK constraints
- **Term based relations** that apply only to a part of a value, e.g. '%Ltd.%' -> '%Limited Company%'

Composite domains can be created of semantically connected fields e.g. city, street, etc. parts of an address stored in more than one columns.

We have some default KBs already installed in the DQS_MAIN on the PRIM server like Country/Region, US-Counties, US-Last names, etc. We can also develop our own KB's.

We can clean the last names of a table using the default KB in the following steps:

Start the DQC and type the server name: .\PRIM.

- On the Administration->Configuration->General settings panel, review the Min Score for Suggestions and Min Score for Auto Correction settings. These settings may be adjusted on a trial and error basis
- On the Knowledge base management panel, review the pre-installed KBs
- Create a new data cleansing project called Lastnames.
- Select the KB: DQS Data and the US_Last Name domain. The activity should be Cleansing
- On the Map page, select the AdventureworksDW2019 database, the DimCustomer table and the Lastname field and map it to the US_Last Name domain
- Start the cleaning. Since we have more than 18000 last names in the DimCustomer table to cleanse, pre-processing will run for ca. 5 minutes, cleansing for ca. 30 s. **WARNING**:[3]
- Check the hard dimensions of Completeness (100%) and Accuracy (98%). After cleansing, a value may be in one of these states:
    - **Correct**: found in the domain values
    - **Corrected**: changed automatically to a domain value because the similarity was above the Min Score for Auto Correction
    - **Suggested**: a new value is only suggested because the similarity was above the Min Score for Suggestions
    - **New**: a similar value is not found among the domain values but the value conforms to the domain rules

[3] If you receive a 'Parallel processing task failed' error during pre-processing, check the DQServerLog.DQS_MAIN.log file in the Log directory of the instance. The error is *'The DELETE statement conflicted with the REFERENCE constraint "B_INDEX_LEXICON_EXTENSION_B_INDEX_LEXICON_FK". The conflict occurred in database "DQS_PROJECTS", table "DQProject1000001.B_INDEX_LEXICON_EXTENSION", column 'TERM_ID''*. Using the SSMS Modify keys panel, set the On Delete action of the B_INDEX_LEXICON_EXTENSION_B_INDEX_LEXICON_FK foreign key constraint to CASCADE.

    - **Invalid**: a new value that contradicts the domain rules

- With the above Min Score settings, the process finds some cases where a correction is suggested and 3 cases where a correction was performed automatically. 328 names were correctly found in the KB.
- Try to Approve a suggested change -> the row will be relocated onto the Corrected pane
- Finally, save the cleansed table containing the LastNameSource and LastNameOutput columns in the original database and process it with T-SQL commands.

Building your own Knowledge Base

We can use an existing set of values in a table field to construct our custom KB. The steps:

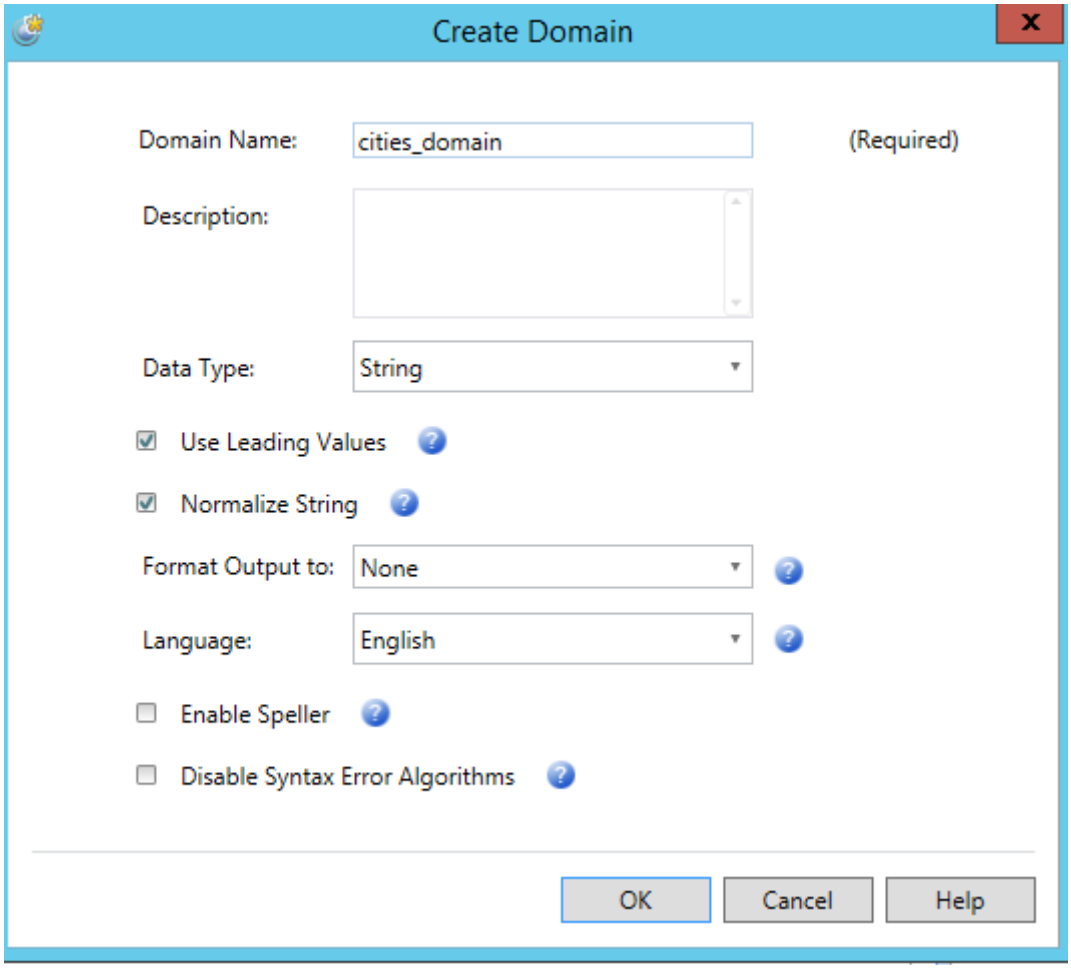
1. Automatic knowledge discovery using the data sample
2. Manual domain management and configuration

We can create a KB in the following steps:

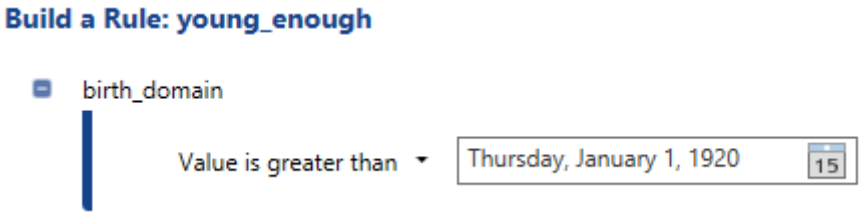
- Create a new view in the DQS_STAGING_DATA database

```
use DQS_STAGING_DATA
go
create view vi_places as
select distinct City, StateProvinceName StateProvince, EnglishCountryRegionName
CountryRegion
from AdventureworksDW2016CTP3.dbo.DimGeography
go
```

- Start the DQS client and create a new KB called places
- Select Knowledge discovery, set the source to the vi_places and create two domains called cities_domain and countryregion_domain using the fields of the same name of the vi_places view



- Start the discovery
- In the slot of Muehlheim set the type to **Error** and manually enter Mühlheim in the 'Correct to' field, *then press Enter*
- Select the new KB from the main menu, and from its popup menu select Domain management
- Manually add a new domain called birth_domain with a simple domain rule:



- Publish the KB
- Select the new KB for management and export it to a file called places.dqs
- Create a new table test_customer in the DQS_STAGING_DATA database from the Dimcustomer table joined to the Dimgeography table on the GeographyKey column
- Change the Birthdate of the first customer to 1925-01-02 and the city to Muehlheim. Change the city of the second customer to Piripócs.
- Use the new 'places' KB to cleanse the test_customer table and review the results

There are also free, independent tools available for data cleaning and master data management e.g. https://openrefine.org/